

A DANGERous Data Challenge

Andrew V. Sutherland

Massachusetts Institute of Technology

DANGER: Data, Numbers, and Geometry, BIRS
April 9, 2026

AI for math

Artificial intelligence and machine learning tools are used in a variety of different ways to help us gain insight and understanding, generate conjectures, and prove theorems.

- Analyze data
 - Search for patterns and correlations (data science)
 - Classification (model training)
- Construct examples
 - Generate extremal/interesting examples ([Funsearch](#), [PatternBoost](#), [AlphaEvolve](#), ...)
 - Extend known or suspected patterns, test conjectures
- Prove theorems
 - Informally ([First Proof](#) solutions by [OpenAI](#) and [DeepMind/Aletheia](#), [Bryan-Elek-Manners-Salafatinos-Vakil](#), [Swaminathan](#), ...)
 - Formally ([AlphaProof](#), [Aristotle](#), [Ax-Prover](#), [AxiomProver](#), [DeepSeek-Prover](#), [Gauss](#), [Gödel-Prover](#), [Kimina-Prover](#), [Seed-Prover](#), ...)

It's all 0s and 1s

Mathematics has two key features that make it especially attractive for AI:

- An inexhaustible supply of noiseless data.
- Computational access to ground truth.

What is data?

In mathematics, data are not observations, they are facts (aka theorems). That the elliptic curve $y^2 + xy + y = x^3 + x^2 - 10x - 10$ has rank 0 is one of 3,824,372 similar facts one can find in the [LMFDB](#). Like all mathematical facts, it can be encoded as a sequence in some fixed alphabet (WLOG, we can assume that alphabet is $\{0, 1\}$).

What is a proof?

A proof (at least in modern mathematics) is a series of permissible steps that allow one to derive a stated conclusion from a stated hypothesis and an agreed list of axioms. Every proof can (at least in principle) be encoded as a sequence in some fixed alphabet.

Warmup example: compositeness

Fact: 91 is composite.

Proof: $91 = 7 \times 13$.

Better proof: $2^{91} \not\equiv 2 \pmod{91}$

The integer 2 witnesses/certifies that 91 is composite.¹

The triple $(91, 7, 13)$ encodes a proof of the compositeness of 91 that can be mechanically translated into a formal proof with a quasi-linear expansion factor.

The pair $(91, 2)$ also encodes a proof of the compositeness of 91 that can be mechanically translated into a formal proof, with a quasi-quadratic expansion factor.

The singleton (91) can also be viewed as encoding a proof of the compositeness of 91 that can be mechanically translated into a formal proof, but with an exponential expansion factor: $2|91 \vee 3|91 \vee 4|91 \vee 5|91 \vee 6|91 \vee 7|91 \vee 8|91 \vee 9|91$.

¹Not all composite numbers have a witness of this type, in general one uses the more general notion of a Rabin-Miller witness.

Primality proving

Fact: 89 is prime.

Non-proof: $89 = 1 \times 89$

Non-proof: $2^{89} \equiv 2 \pmod{89}$

Proof: $2^{88} \equiv 1 \pmod{89} \wedge 2^{44} \not\equiv 1 \pmod{89} \wedge 2^8 \not\equiv 1 \pmod{89}$

$\wedge 89 - 1 = 88 = 2^3 \times 11 \wedge 2 \text{ is prime} \wedge 11 \text{ is prime}$

Better proof: The point $P := (6, 45)$ on the elliptic curve $y^2 = x^3 + 13x^2 + x$ satisfies $2^4 P \neq 0$ and $2^5 P = 0$, with $2^5 > q + 1 + \lfloor 2\sqrt{q} \rfloor$ where $q = \lfloor \sqrt{p} \rfloor = 9$.

The **Pomerance triple** (89, 13, 6) encodes a proof of the primality of 89 that can be mechanically translated into a formal proof, with a quasi-quadratic expansion factor.

Pomerance proofs are the **gold standard** of primality proofs; every other type (Pratt, ECPP, AKS, ...) is more complicated and has a larger expansion factor.

Pomerance proofs

For a positive integer p , define $k_p := \lceil \log_2(q + 1 + 2\sqrt{p}) \rceil$ where $q = \lfloor \sqrt{p} \rfloor$.

Definition

A triple of integers (p, A, x_0) with $x_0, A \in [0, p-1]$ and $A^2 \not\equiv \pm 2 \pmod{p}$ is a **Pomerance triple** if there exist integers $B, y_0 \in [0, p-1]$ such that $P := (x_0, y_0)$ is a solution to $E: By^2 = x^3 + Ax^2 + x$, with $2^{k_p}P = 0$ and $2^{k_p-1}P \neq 0$.²

Theorem

If (p, A, x_0) is a Pomerance triple with $p > 2$ odd, then p is prime.

Proof.

$A \not\equiv \pm 2 \pmod{p}$ ensures E is non-singular modulo any odd divisor q of p .

If p is not prime then we may choose $q \leq \sqrt{p}$. The point $P = (x_0, y_0)$ has order 2^{k_p} which is larger than $\#E(\mathbb{F}_q) \leq q + 1 + 2\sqrt{q}$, a contradiction. □

²The inequality $2^{k_p-1}P \neq 0$ must hold over $\mathbb{Z}/q\mathbb{Z}$ for every $q|p$.

Pomerance proofs

Note that $By_0^2 = x^3 + Ax^2 + x$ uniquely determines the square class of B in $(\mathbb{Z}/p\mathbb{Z})^\times$, which uniquely determines the isomorphism class of $By^2 = x^3 + Ax^2 + x$ for a given A , so we can take B to be any representative of its square class, and this determines y_0 .

But in fact we never need to know the values of B and y_0 !

Given a projective point $P = (x_1 : y_1 : z_1)$ on $By^2z = x^3 + Ax^2z + xz^2$, we can compute $2P = (x_2 : y_2 : z_2)$ by letting $C := (A + 2)/4$ and applying

$$u = (x_1 + z_1)^2$$

$$v = (x_1 - z_1)^2$$

$$w = u - v$$

$$x_2 = uv$$

$$z_2 = w(v + Cw)$$

Certifying a Pomerance triple

```
1 def pp_verify(p, A, x0):
2     from math import gcd, isqrt
3
4     if p < 5 or p % 2 == 0:
5         return False
6
7     q = isqrt(p)
8     k = (q + 1 + isqrt(4*q)).bit_length()
9
10    # p not prime ==> prime divisor < q
11    # least k such that 2^k > q + 1 + 2*sqrt(q)
12
13    if gcd(A * A - 4, p) != 1:
14        return False
15    # singular
16
17    X, Z = x0 % p, 1
18    Zprev = None
19
20    # precompute C = (A + 2)/4 mod p for doubling formula
21    C = ((A + 2) * ((p + 1) // 4 if p % 4 == 3 else (3*p + 1) // 4)) % p
22    for i in range(k):
23        Zprev = Z
24        U, V = (X+Z)*(X+Z) % p, (X-Z)*(X-Z) % p
25        W = U - V
26        X, Z = U*V % p, W*(V+C*W) % p
27
28    # if p|Z and (Zprev,p)=1 then (p,A,x0) is valid
29    return Z % p == 0 and gcd(Zprev, p) == 1
```


Demonstration

Code and data are available in the following GitHub repo:

<https://github.com/AndrewVSutherland/DANGER3>

```
python3 vpp.py 89 13 6
```

```
python3 lean_vpp.py 89 13 6 >pp89.lean
```

Online Lean4/Mathlib calculator available at:

<https://live.lean-lang.org/>

The landscape of Pomerance triples

Theorem

Pomerance triples (p, A, x_0) exist for every prime $p > 3$.

For any fixed p , there are on the order of $\sqrt{p}$ integers $A \in [0, p-1]$ that appear in a Pomerance proof for p ,³ and for each such A there are $2^{k_p-1} \approx \sqrt{p}$ possibilities for x_0 . Thus every prime $p > 3$ appears in roughly p distinct Pomerance triples.

Given a prefix (p, A) of a Pomerance triple, it is easy to find an x_0 for which (p, A, x_0) is a Pomerance triple (this takes quasi-quadratic expected time).

For a general prime p , no efficient algorithm is known for finding a Pomerance triple (p, A, x_0) . The fastest known general algorithm simply picks A 's at random and tries to extend (p, A) to (p, A, x_0) (this will fail or succeed in quasi-quadratic expected time).

³"On the order of" ignores factors that are $O(\log p)$.

Data challenge

Challenge: Given a (potentially large) prime p , generate a Pomerance triple (p, A, x_0) .

Resources available at <https://github.com/AndrewVSutherland/DANGER3>:

- [vpp.py](#): Python program to test candidate triples.
- [pp10.txt](#): the 236,264 Pomerance triples with $p < 2^{10}$.
- [pp20.txt](#): one Pomerance triple for each of the 82,023 primes $3 < p < 2^{20}$.
- [pp12.txt.gz](#): the 3,083,880 Pomerance triples with $p < 2^{12}$.
- [pp24.txt.gz](#): one Pomerance triple for each of the 1,077,869 primes $3 < p < 2^{24}$.
- [pp16A.txt.gz](#): the 4,713,069 prefixes (p, A) of Pomerance triples with $p < 2^{16}$.

Resources available elsewhere (links available in the GitHub repo):

- [pp14.txt.gz](#): the 43,307,624 Pomerance triples with $p < 2^{14}$ (646MB).
- [pp16.txt.gz](#): the 598,705,640 Pomerance triples with $p < 2^{16}$ (9887MB).
- [pp28.txt.gz](#): one Pomerance triple for 14,630,841 primes $3 < p < 2^{28}$ (386MB).